

장준환

반복되는 문제를 개선하고, 신뢰할 수 있는 파이프라인을 만드는 엔지니어

010-7356-9008

neon9008@gmail.com

GitHub

Tistory

학력/교육

삼성청년SW·AI아카데미 (SSAFY) 15기	2026.01 -
현대오토에버 모빌리티 SW 스쿨 클라우드 2기 수료	2025.04 - 2025.11
건국대학교 스마트운행체공학과 졸업 (4.2/4.5)	2017.03 - 2024.02

경력

CJ ENM 아티스트 사업부 데이터 계약직	2024.03 - 2024.11
대중음악웹진 IZM 편집장	2019.03 - 2024.09

자격증

정보처리기사	2026.06.12
네트워크관리사 2급	2025.09.30
SQLD (SQL 개발자)	2024.11.17

어학

OPIc / IM1	2025.03.13
TOEIC / 770	2025.02.23

수상

삼성청년SW·AI아카데미 1학기 성적우수상	2026.06.26
2019년 2학기 건국대학교 Dean's List 수상	

About Me

반복되는 문제를 개선하고, 신뢰할 수 있는 파이프라인을 만드는 엔지니어

평론과 데이터 업무에서 반복되는 수집과 정리를 개선하고 싶었습니다. 학부 시절 배운 데이터베이스와 인공지능 지식을 기반으로 지표 수집을 자동화하며 개발자의 길을 선택했습니다. 관찰력과 분석력으로 불편을 발견하고 개선합니다.

- **팀원으로서** 진행 상황과 막힌 부분을 공유하고, 의견이 다를 때는 근거를 바탕으로 조율합니다.
- **팀장으로서** 목표와 우선순위를 명확히 하고, 팀원들의 의견과 어려움을 듣고 역할과 일정을 조율합니다.

다루는 기술



Python ■■■■■

수집 스크립트와 정제 로직을 구현하고, 실패 판정 로직을 테스트했습니다.



Django ■■■□□

beautalk의 인증, 커뮤니티와 추천 이력 API를 구현했습니다.



FastAPI ■■■□□

digem의 데이터 조회 API와 서비스 백엔드를 구현했습니다.



PostgreSQL ■■■□□

데이터 모델과 조회를 설계하고, 행 잠금으로 동시 쓰기를 처리했습니다.



Redis ■■■□□

이음길에서 변경 순서를 채번하고 실시간 동기화에 활용했습니다.



Docker ■■■■□

서비스별 컨테이너와 Compose 실행 환경을 구성했습니다.



Kubernetes ■■■■□

Finch를 단일 노드 k3s로 옮기고 Helm으로 배포 구성을 관리했습니다.



AWS ■■■■□

digem의 EC2 운영과 Lambda·S3 기반 구성 전환을 경험했습니다.



GitHub Actions ■■■□□

수집 예약 실행과 배포 자동화 작업을 구성했습니다.



Jenkins ■■□□□

프로젝트 CI/CD 파이프라인 구성에 사용했습니다.



Prometheus ■■■□□

Finch의 애플리케이션 지표를 수집하도록 구성했습니다.



Grafana ■■■□□

Finch에서 지표와 로그를 확인하는 운영 화면을 구성했습니다.



Git ■■■■■

브랜치와 커밋으로 변경을 관리하고 팀 작업을 통합했습니다.



Jira ■■■■■

팀 프로젝트의 작업과 진행 상황을 공유했습니다.

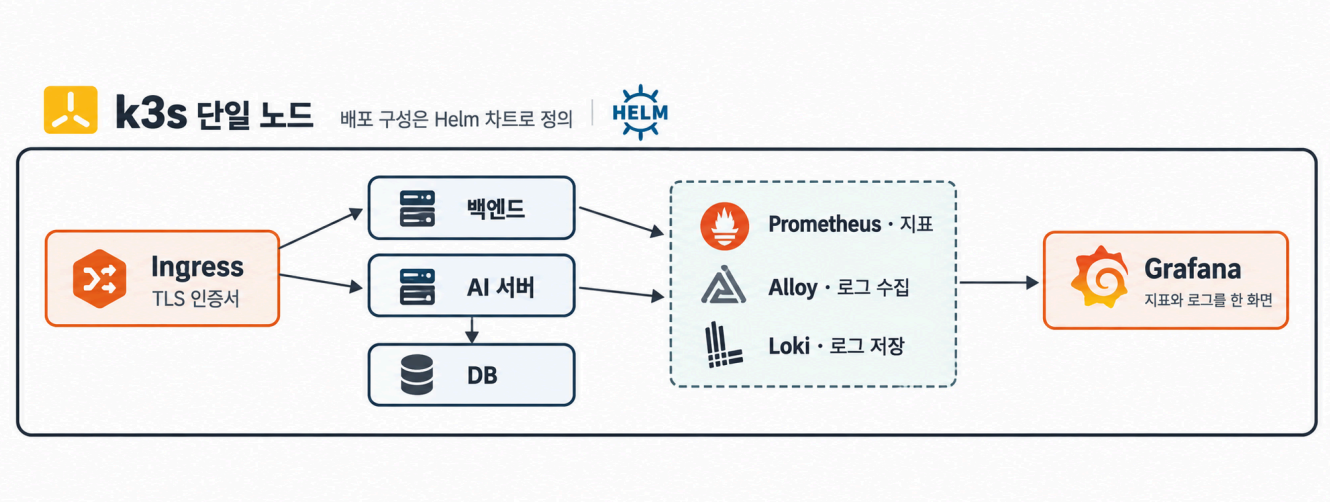
① Finch : 가상 계좌 기반 모의투자 서비스

Github | 시연 영상

5인 / 인프라와 운영 환경 / 2026.08 ~ 09

공시와 뉴스에서 내 종목에 필요한 정보를 골라내는 모의투자 서비스입니다. 서버 한 대에서 애플리케이션을 배포하고 지표와 로그를 확인할 수 있도록 운영 환경을 구성했습니다.

아키텍처



Ingress가 요청을 전달하고, 서비스 지표와 컨테이너 로그는 관측 도구를 통해 확인합니다. 애플리케이션 내부 구현은 생략하고 배포·운영 구성을 중심으로 표현했습니다.

기술 선택 이유

k3s

서버 한 대 규모에 맞는 경량 Kubernetes 배포판을 선택했습니다. 컨테이너 교체와 준비 상태 관리를 배포 환경에서 다루기 위해 사용했습니다.

Helm

배포 구성을 차트로 정의하고 환경별 설정을 관리하기 위해 사용했습니다. 애플리케이션 배포에 필요한 설정을 함께 다뤘습니다.

Ingress

외부 요청의 진입점과 TLS 인증서 처리를 한곳에서 관리하기 위해 사용했습니다.

Prometheus

애플리케이션 지표를 수집하고 엔드포인트별 응답 시간 분포를 확인해 병목을 좁히기 위해 사용했습니다.

Alloy · Loki

애플리케이션에 로그 적재 코드를 추가하지 않고 컨테이너 로그를 수집·저장하기 위해 선택했습니다.

Grafana

수집한 지표와 로그를 한 화면에서 확인하고 서비스의 동작 상태를 살펴보기 위해 사용했습니다.

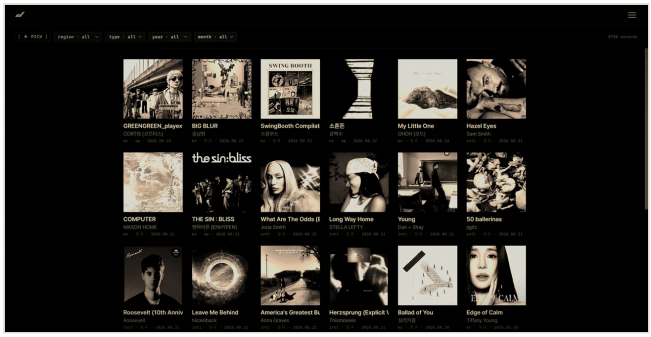
2 digem : 글로벌 음악 소식 자동 수집 가공 아카이브

Github | 사이트 이동 PW: digem123! 1인 / 풀스택 / 2025.08 ~ 진행 중

해외 음악 칼럼과 앨범 정보를 수집하고 AI로 번역해 한국어 아카이브로 제공하는 **개인 ETL 서비스**입니다. 수집부터 저장과 조회까지 구현하고, 실행 주기와 작업 규모에 맞춰 세 실행 구조를 거치는 동안 운영 환경을 두 차례 전환했습니다.

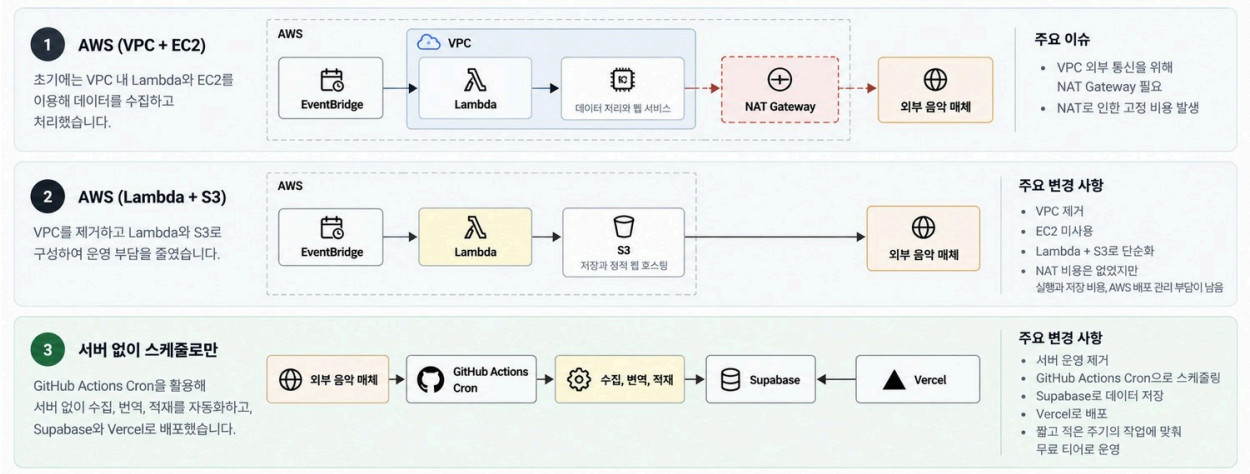


칼럼 페이지



앨범 페이지

아키텍처 전환



파이프라인 구성

1 수동 크롤러를 자동 ETL 파이프라인으로 전환

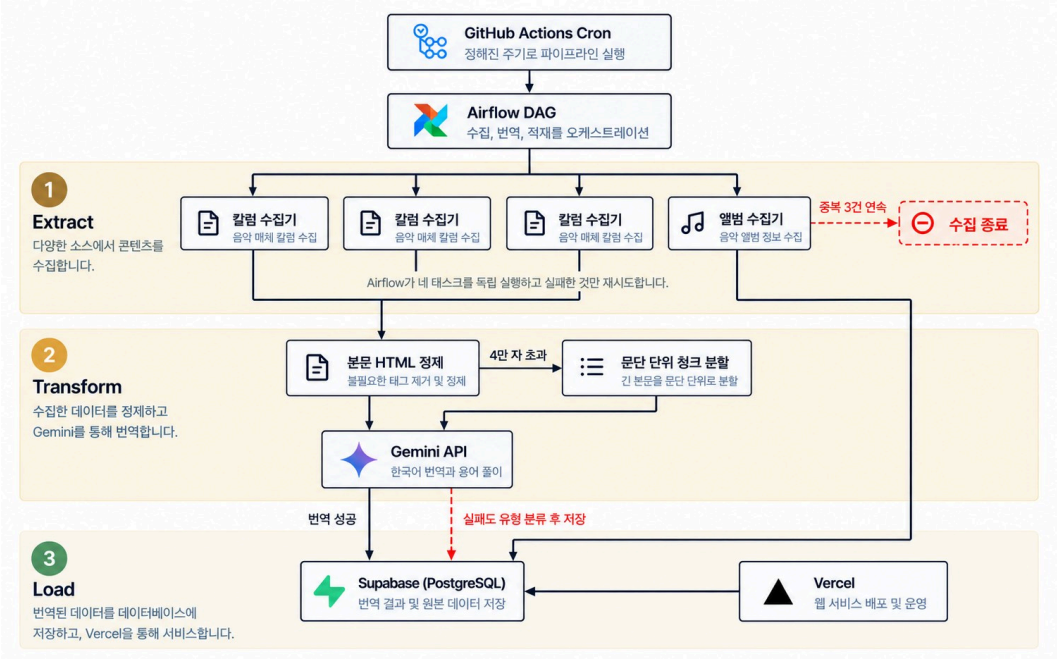
이전 직장에서 만든 수집 스크립트가 출발점이었습니다. 수동 실행이라 실행 주체가 없으면 신규 데이터 수집이 중단됐고, 파일 안에서 순차로 하나씩 실행돼 스크래퍼 하나가 실패하면 처음부터 다시 돌려야 했습니다.

- GitHub Actions cron으로 하루 한 번 Python 진입점을 직접 실행합니다.
- 매체별 예외 처리로 실패를 격리하고, 나머지 수집을 마친 뒤 종료 코드로 실패를 알립니다.
- 매체마다 RSS 제공 여부, HTML 구조, 페이지네이션이 달라, 부모 클래스에 **수집, 파싱, 정제, 저장의 공통 흐름**을 두고 매체별 차이만 개별 구현했습니다.

2 불필요한 수집과 번역 API 호출 차단

앨범 수집기가 매 실행마다 최신 목록 전체를 순회했고, 번역에 실패한 기사를 건너뛰면 다음 실행 때 같은 기사를 다시 수집해 API 호출이 반복됐습니다.

- 중복 데이터가 **연속 3건 나오면 수집을 중단**하는 조기 종료로 불필요한 조회를 줄였습니다.
- 번역 실패 기사도 **원문과 메타데이터를 실패 상태로 저장**해 재수집을 막았습니다.
- 실패를 다음 행동에 따라 분류해 저장했습니다. 재시도, 대기, 분할과 사람 확인을 판정하며 자동 재번역은 후속 작업입니다.



3 목록 조회 성능 최적화

목록 화면에서 select(*)로 사용하지 않는 원문과 번역문까지 함께 조회해 로딩 시간이 길어졌습니다.

- 목록은 **필요한 메타데이터만 명시적으로 조회**하도록 바꿨습니다.
- 상세 페이지의 원문과 번역문은 토글 전환 시 **지연 조회**하도록 분리했습니다.
- 캐시 비활성화 측정: 200kB → 16.8kB, 3.14초 → 252ms. 반복 횟수와 네트워크 조건은 미기록입니다.

4 음악 평론 문맥을 반영한 AI 번역 파이프라인 설계

음악 평론에는 일반 독자가 낯선 장르와 연주 기법 용어가 자주 등장해 직역만으로는 이해하기 어렵고, 긴 기사는 모델의 입출력 한도를 넘길 가능성이 있었습니다.

- 프롬프트에 **어려운 평론 용어는 짧은 한국어 풀이를 함께 제공**하도록 규칙을 설계했습니다.
- Gemini 모델의 실제 처리 여유를 측정해 **최대 청크를 4만 자로 설정**하고, 초과 시 문단 단위로 분할해 번역한 뒤 결합했습니다.
- 영문 병기는 **백틱으로 감싸 식별 가능한 마크업**으로 만들고, 후처리에서 두 번째 이후 구간만 삭제해 중복을 제거했습니다.
- 매체별 고정 노이즈 문구를 미리 조사해 **규칙 기반 후처리**로 정제했습니다.

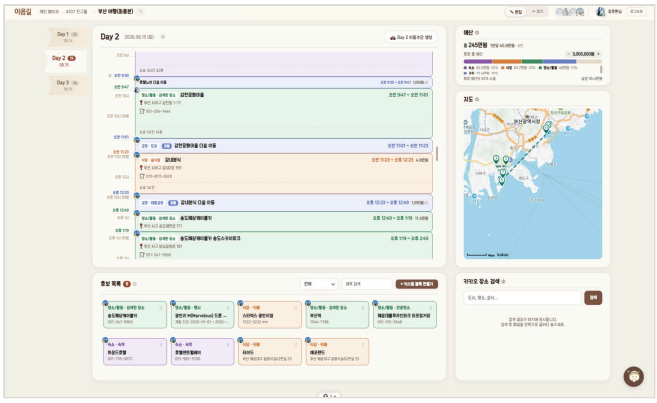
> 1826년, 니세포르 니엡스Nicéphore Niépce라는 프랑스 발명가가 그의 저택 창밖으로 카메라 옵스큐라camera obscura; 카메라의 원조가 되는 상자형 암상자를 겨냥했고, 약 6주에 걸쳐 건물과 시골 풍경을 담은 이미지를 만들어냈습니다. 기본적으로 모든 사람들은 그것이 세계에서 가장 오래 살아남은

3 이음길 : 여행 일정을 함께 편집하는 협업 보드

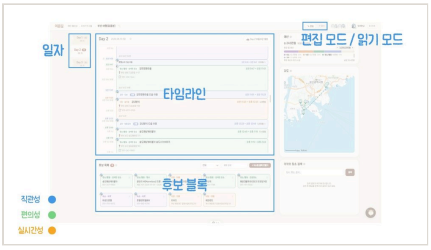
Github

6인 / 백엔드 / 2026.07 ~ 08 (4주)

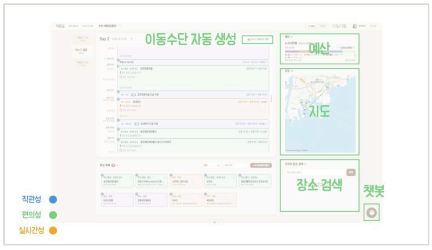
여러 사용자가 여행 일정 보드의 블록을 옮기고 **실시간으로 공동 편집**하는 서비스입니다. 저는 변경 처리와 유실 복구를 맡았습니다. 변경과 저널을 한 트랜잭션에 저장하고, 커밋 후 알림을 보내도록 구현했습니다. 놓친 변경은 번호 구간으로 다시 요청하게 했으며, 같은 블록의 동시 쓰기와 잠금 경합을 점검하고 보완했습니다.



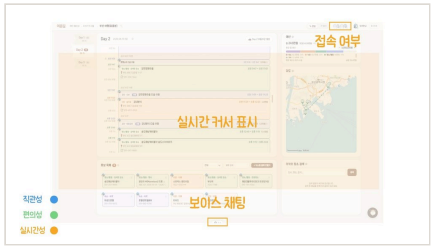
일정 보드 화면



직관성



편의성



실시간성

실시간 동기화 코어

1 두 계층의 락이 얽히는 교착 경로 차단

프로젝트 락을 커밋까지 연장한 뒤, JVM 락과 DB 행 락을 반대 순서로 획득하는 경로를 시연 전에 발견했습니다. 실제 교착 발생을 재현한 것이 아니라, 교차 대기가 가능한 실행 경로를 찾아 차단한 작업입니다.

- PostgreSQL 감지기는 JVM 락까지 보지 못합니다. 상한 없는 대기를 막기 위해 락 획득 시간과 순서를 함께 보완했습니다.

- 1차로 **락 획득에 상한(tryLock)**을 걸어 무한 대기를 끊었습니다.
- 2차로 **flush를 락 획득보다 앞세워** 두 락의 획득 순서를 한 방향으로 통일하고 원인을 제거했습니다.
- 실제 교차 교착은 재현하지 못했습니다. 장시간 락 점유 대리 테스트로 후속 요청의 503 종료와 락 반환을 확인했습니다.

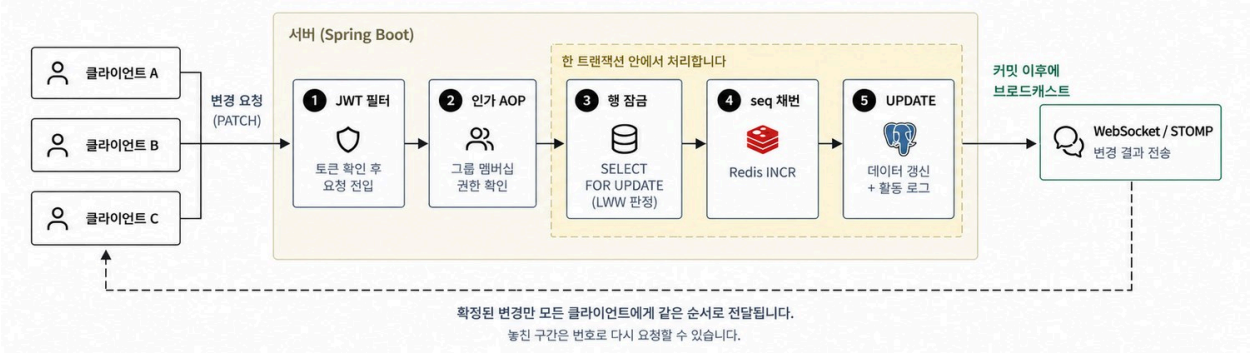
2 동시 쓰기 유실 해소 (블록 행 잠금)

두 사람이 같은 블록의 서로 다른 필드를 동시에 고치면, 행 단위로 덮어쓰던 구조에서는 나중 요청이 앞선 변경을 통째로 지웠습니다.

- 같은 블록의 쓰기를 DB 행 잠금으로 직렬화했습니다. 다른 블록의 병렬 처리는 유지했습니다.
- 삭제는 **tombstone으로 남겨**, 뒤늦게 도착한 변경을 410으로 구분했습니다.

3 순서 보장과 유실 복구

- Redis **INCR로 시퀀스를 채번**하고 DB 기준으로 리시드해 재시작 후에도 번호가 어긋나지 않게 했습니다.
- 브로드캐스트는 처음부터 커밋 이후에만 보내도록 설계해, 저장되지 않은 변경이 알림으로 나가지 않게 했습니다.
- 클라이언트가 놓친 변경을 **번호 구간으로 재요청**하는 API를 함께 뒀습니다.



4 그룹 수와 무관한 조회 횟수 설계

- 대시보드 스냅샷 API를 **쿼리 4회 고정**으로 설계했습니다. 블록, 멤버, 활동로그를 따로 조회하면 참여자 수에 따라 늘어납니다.
- 멤버십 검증에서 복합키 엔티티의 exists 쿼리가 두 테이블을 조인해 전 컬럼을 읽고 있어 **JPQL count로 교체해 조인을 제거**했습니다.
- 그룹 목록은 ID를 모아 IN 절과 JOIN FETCH로 일괄 조회해 처음부터 쿼리 3회로 구성했습니다.

5 실시간 채널의 인증과 인가 점검

- CONNECT 시 JWT 인증, SUBSCRIBE와 SEND 목적지 인가, 세션 레지스트리, 하트비트를 구성했습니다.
- 이후 점검에서 **세션 인증에 만료가 없어 탈퇴 후에도 실시간 수신이 가능한 문제**를 찾아 고쳤습니다.
- **보이스 시그널 대상을 검증하지 않아 임의 멤버에게 오퍼를 보낼 수 있던 문제**도 함께 고쳤습니다.

6 반복 검증의 AOP 분리와 회귀 테스트

- **@GroupMember 애노테이션 하나로** groupId, projectId, blockId 어느 파라미터에서든 그룹을 역추적해 검증합니다.
- 404와 403의 순서를 지켜 **존재 여부와 권한 여부를 구분**했습니다.
- 회귀 테스트 약 1,070행으로 동시성, tombstone, 커밋 후 알림, 잠금 경합과 만료, 인가와 시퀀스 리시드를 검증했습니다. 응답 성공 여부뿐 아니라 최종 저장 상태도 대조했습니다.

4 beautalk : 화장품 추천 챗봇 서비스

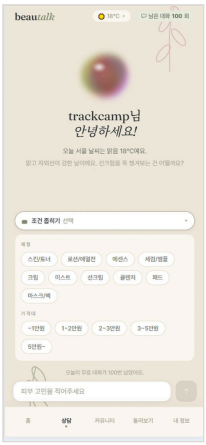
Github

2인 / 백엔드, AI / 2026.05 ~ 06

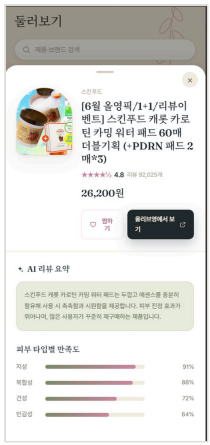
수집한 화장품 데이터에서 사용자의 피부 고민, 가격대, 원하는 제형에 맞는 제품을 추천하는 서비스입니다. 백엔드에서 **OAuth와 JWT 인증**, 찜과 추천 이력, 커뮤니티, 날씨 API 연동을 구현했습니다. 데이터와 AI 영역에서는 **상품 크롤링과 정제, 리뷰 요약, 임베딩 생성, SQL 필터와 RAG를 결합한 추천 기능**을 담당했습니다.



대시보드

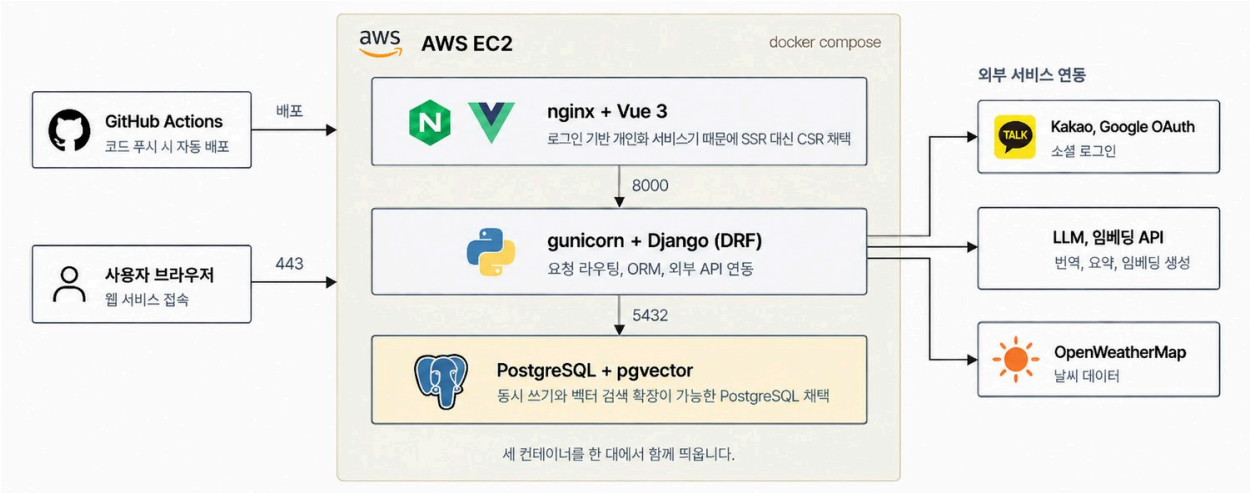


추천 챗봇



제품 상세

서비스 구조



추천 로직

1 추천 데이터 품질 개선

- 103개 상품 중 리뷰 요약이 비어 있던 40건을 리뷰 기반 LLM 요약으로 보완했습니다.
- 별도 제형 컬럼은 상품명과 카테고리 규칙으로 306종 중 281종을 분류했습니다. 요약 보완과는 다른 점검 범위입니다.

2 SQL과 RAG를 결합한 추천 구조 설계

의미 유사도로 후보를 찾던 구조에서 2~3만원대 로션 요청에 다른 제형과 5,700원 제품이 섞였습니다.
임베딩 문장과 추천 프롬프트에 가격이 없었고, 제형을 필터링할 컬럼도 없었습니다.

- 가격과 제형을 규칙으로 추출하고, 제형 컬럼을 보완해 SQL로 조건에 맞는 후보를 먼저 선별했습니다.
- PostgreSQL의 **pgvector** 확장으로 선별된 후보 안에서 사용자 요청과 의미가 가까운 상품을 정렬했습니다.
- 반환된 상품 ID는 DB에서 존재 여부를 확인했습니다. 조건 제한은 후보 구성 단계이며 재조회가 조건을 재검증하지는 않습니다.

3 추천 LLM 호출 24초 → 3.6초 단축

추천 응답이 약 25초 걸려 사용자 경험을 해치는 문제가 있었습니다.

- **LangSmith 트레이싱**으로 추천 과정의 입력, 출력, 단계별 지연 시간을 기록했습니다.
- 임베딩 1초, SQL 조회 0.1초였지만 **LLM 호출이 약 24초로 전체 지연의 96%**를 차지함을 확인했습니다.
- 같은 프롬프트로 추론 2종과 비추론 3종을 비교했습니다. 후보 15개 고정, 모델별 1회 측정한 LLM 호출 시간이 24초에서 9.1초, 모델 교체 후 3.6초로 줄었습니다. 전체 응답 시간은 아닙니다.
- 모델명은 **환경 변수로 분리**해 운영 중에도 빠르게 교체하거나 이전 모델로 되돌릴 수 있도록 구성했습니다.

Name	Input	Output	Error	Start Time	Latency
gms_call	user: 이번엔 크림을 추천해줘	["복합성 피부의 건조함을 해...		6/25/2026, 6:5...	0.95s
resolve_constraints	user: 이번엔 크림을 추천해줘	{"categories": []"forms": [...		6/25/2026, 6:5...	0.00s
gms_call	user: 지금까지의 대화를 바...	["{"content": "복합성 피부...		6/25/2026, 6:5...	4.45s
recommend_candidates	assistant: 복합성 피부에 적...	[["Product object (519613...		6/25/2026, 6:5...	1.66s
gms_call	user: 내 피부타입에 맞는 스...	["{"content": "복합성 피부...		6/25/2026, 6:5...	0.92s
resolve_constraints	user: 내 피부타입에 맞는 스...	{"categories": []"forms": [...		6/25/2026, 6:5...	0.00s
gms_call	user: 스킨/토너 제품을 1~2...	["{"content": "스킨/토너 제...		6/25/2026, 6:5...	1.65s
resolve_constraints	user: 스킨/토너 제품을 1~2...	{"categories": []"forms": [...		6/25/2026, 6:5...	0.01s
gms_call	user: 지금까지의 대화를 바...	["{"content": "지성 피부에 ...		6/25/2026, 6:5...	4.42s
recommend_candidates	assistant: 알겠습니다. 지성...	[["Product object (ac6d0...		6/25/2026, 6:5...	2.11s
gms_call	user: 스킨/토너 제품을 1~2...	["{"content": "알겠습니다. ...		6/25/2026, 6:5...	1.11s

단계마다 걸린 시간이 기록됩니다. 규칙으로 조건을 뽑는 `resolve_constraints`는 0.00초, LLM을 부르는 `gms_call`은 초 단위로 걸립니다.

4 날씨를 대화 맥락으로만 쓰도록 설계

- OpenWeatherMap으로 현재 날씨를 읽어 **제품을 권하는 문장이 아니라 대화의 맥락**으로 넘겼습니다. 맑으면 선케어를, 비가 오면 워터프루프를 **자연스럽게 언급하도록** 안내만 합니다.
- 같은 날씨를 매 대화마다 조회하지 않도록 **30분 캐시**를 두었습니다.
- API 키가 없거나 조회에 실패하면 **빈 문자열을 돌려 대화는 그대로 진행**됩니다. 부가 기능이 본체를 멈추게 하지 않도록 했습니다.

```
_WEATHER_SKIN_HINT = {
    'clear':  '맑고 자외선이 강합니다. 선케어·수분 제품을 자연스럽게 언급하면 좋습니다.',
    'rain':   '비가 내립니다. 워터프루프나 클렌징 제품을 언급해도 좋습니다.',
    'snow':   '추운 날씨입니다. 장벽 강화·보습 크림을 자연스럽게 언급하세요.',
    'clouds': '흐리거나 대기질이 좋지 않습니다. 클렌징이나 자외선 차단을 가볍게 언급할 수 있습니다.',
}
```

날씨 상태마다 어떤 맥락을 넘길지 규칙으로 정해 두었습니다.

5 앱별 테스트를 빌드에서 자동 수행

기능을 고칠 때마다 권한 검사나 중복 요청 처리가 조용히 깨질 수 있는데, 손으로 확인하면 매번 같은 경로만 보게 됩니다.

- 계정, 커뮤니티, 대화, 상품 네 개 앱에 테스트를 두고 **총 65개를 빌드 때 자동 수행**하도록 구성했습니다.
- **다른 사용자의 글을 수정하거나 삭제할 수 없는지** 검사해 권한 우회를 막았습니다.
- 좋아요를 두 번 눌러도 결과가 같은지 **멱등성**을 확인하고, 취소할 것이 없을 때는 404를 주도록 했습니다.
- 없는 리소스에는 404, 권한이 없으면 403을 주는지 **에러 코드를 구분해** 검사했습니다.

5 IZM : 20년 레거시 음악 웹진 현대화

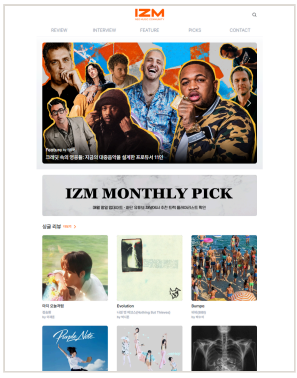
과거 사이트 | 현재 사이트

2024.01 ~ 05

편집장으로 활동할 시절, 2001년 개설된 웹진(Classic ASP 기반)이 해킹 피해를 입어, 사이트를 복구하고 리뉴얼하면서 인프라와 데이터 구조까지 다시 설계한 프로젝트입니다. **PM**으로서 외부 개발사와의 미팅을 주도하고, 매주 스크럼 회의를 통해 요구사항을 전달하며 일정과 진행 상황을 관리했습니다. 요구를 말로 전달하면 해석이 갈리기에 **PPT로 레이아웃의 경우의 수를 하나씩 만들었고**, 학부 데이터베이스 수업에서 배운 지식으로 **ERD를 그려 개선 스키마가 담긴 제안서**를 작성해 전달했습니다.



기존 사이트



개편 사이트

1 콘텐츠와 아티스트의 연결 구조 개선

기존에는 콘텐츠 하나에 아티스트를 한 명만 연결할 수 있어, 협업곡과 컴필레이션을 등록하려면 우회책으로 아티스트 이름을 이어붙인 레코드를 새로 생성해야 했습니다.

- 콘텐츠와 아티스트의 직접 참여 관계는 유지하고, **관련 아티스트를 위한 별도 N:M 관계를 추가**했습니다.
- 관련 아티스트 태그를 기반으로, 아티스트 페이지에서 관련된 아티스트, 언급된 콘텐츠까지 탐색할 수 있도록 개선했습니다.

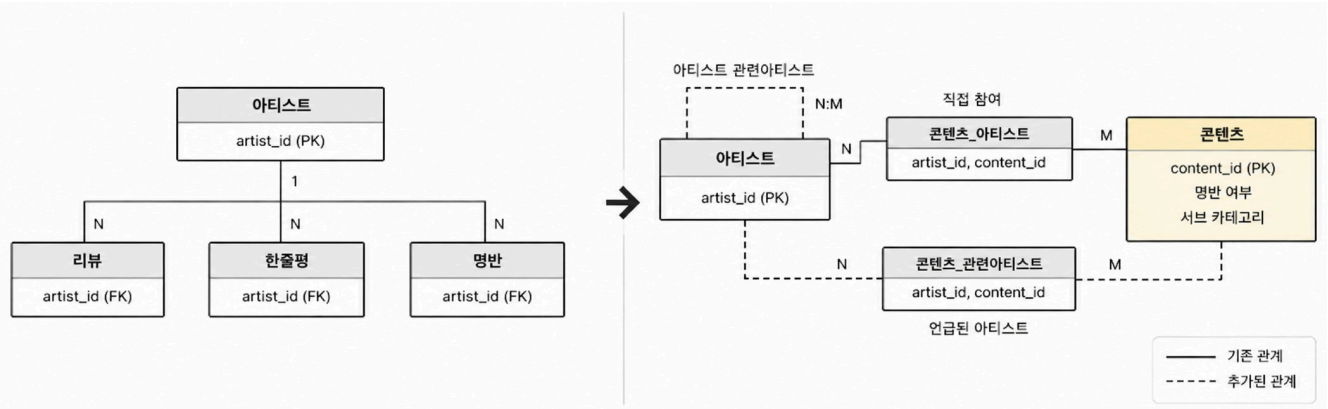


개선 전: 같은 아티스트가 참여 조합마다 다른 레코드 → 개선 후: 참여자를 각각 연결하고 피처링까지 구분

2 중복 등록을 줄이는 운영 모델 구축

기존에는 '리뷰', '한줄평', '명반'이 '아티스트'라는 테이블을 참조하는 별도 테이블로 존재했고, 같은 앨범도 리뷰용, 한줄평용, 명반용을 따로 등록해야 하는 불편함이 있었습니다.

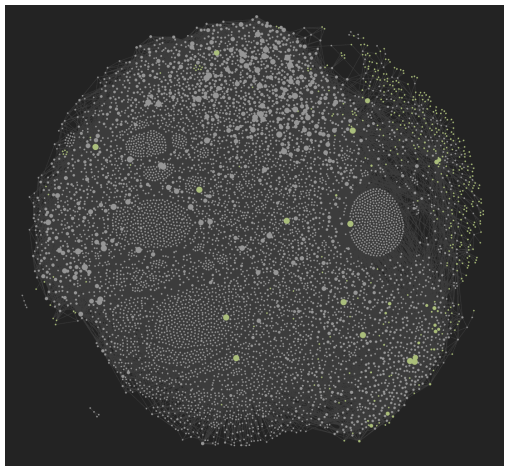
- 모든 글은 **'콘텐츠' 테이블로 통일**하고 명반 선정 여부와 서브 카테고리를 **속성으로 관리**하도록 통합했습니다.
- 중복 등록을 방지하고, 하나의 콘텐츠를 목적에 따라 재사용할 수 있도록 개선했습니다.



6 세컨드 브레인 : 음악 지식 베이스

1인 / 2026.07 ~ 진행 중

음악 칼럼을 읽고 **용어를 정리하고 인사이트를 뽑아 사고를 넓히려고** 만들었습니다. 그 전에도 자료를 찾아 노션에 모아뒀지만, 필요할 때마다 다시 찾아 읽어야 했고 예전에 잘못 적어둔 것을 발견하기 어려웠습니다. 2026년 7월부터 **AI를 적극 활용하되 확실한 정보만 적재하고, 충돌하는 내용은 검증하며, 기존 자료의 오류까지 검출하는 구조**로 다시 만들었습니다. 칼럼 하나를 읽으면 명제 단위로 쪼개 아티스트, 레이블, 장르에 걸어둡니다. 지금까지 **노트 7,226개**를 쌓았고 그 사이에 **연결 35,917개**가 생겼습니다.



노트 사이 연결을 그린 그래프

규칙과 판정 기준

1 규칙을 먼저 정하고 AI는 그 안에서만

AI로 노트를 대량 생성하면 빠르지만, 근거 없는 서술이 섞여도 그대로 쌓입니다. 잘못된 사실은 나중에 다른 노트와 충돌해야 겨우 드러납니다.

- 볼트 정책 문서에 **"문서가 존재하고 대상이 정확히 특정된 경우에만 작성한다. 없거나 불명확하면 건너뛰거나 보류한다"**를 최우선 기준으로 뒀습니다.
- **"근거 없는 평가나 출처 밖의 추측으로 내용을 대체하지 않는다"**를 함께 명시하고, 원문 URL을 노트마다 남기게 했습니다.
- 노트 하나를 완료로 표시하기까지 순서를 정해 뒀습니다. **대상이 맞는지 확인하고, 원문을 다 읽고, 사실의 누락과 왜곡부터 검토한 뒤, 메타데이터와 링크를 걸고, 기존 앨범과 리뷰가 지워지지 않았는지 확인**합니다. **여덟 단계를 모두 통과한 뒤에만 완료로 표시**합니다.

2 동명이인 판정을 사람에게 넘기는 기준

아티스트 이름은 겹칩니다. 자동으로 연결하면 조용히 틀린 관계가 쌓이고, 사람이 전부 확인하면 규모를 감당할 수 없습니다.

- 정책에 **"동음이의어나 국적이 불명확한 대상은 자동 링크나 스텁을 만들지 않고 검토 목록에 기록한다"**를 규칙으로 뒀습니다.
- 판정 스크립트가 문서 존재 여부, **동음이의 문서인지, 음악가가 아닌지, 한국 아티스트가 영문명으로 들어와 있는지**를 확인해 다섯 가지로 분류합니다.
- 스크립트는 **판정 표만 출력하고 노트는 건드리지 않습니다.** 확실한 것과 애매한 것을 나눠, 애매한 것만 사람이 봅니다.

